現場最前線から教える データサイエンス４

データサイエンティストの開発技術

2025年度　〇〇大学

日本IBMシステムズ・エンジニアリング株式会社
DXセンター データサイエンスラボ
川口 英俊

自己紹介

- 講師名: 川口 英俊 (かわぐち ひでとし)
- 所属:
  - 日本IBMシステムズエンジニアリング(株) DXセンター データサイエンスラボ（Experienced Data Scientist)
  - 情報処理学会　論文誌ジャーナル編集委員会 知能グループ（編集委員, メタ査読者)
- 学位: 博士（情報科学)
- 経歴
  - 2010年-2014年 広島工業大学 情報学部 知的情報システム学科 (学士)
  - 2014年-2016年 東京工業大学 総合理工学研究科 知能システム科学専攻博士前期課程 (修士)
  - 2016年-2023年 日本電信電話株式会社(現: NTT株式会社)
    - 在籍中2019年-2023年: 北陸先端科学技術大学院大学 先端科学技術研究科 先端情報科学プログラム 博士後期課程(博士)
  - 2023年 7月より現職
- 専門: 機械学習, データ分析, 自然言語処理, ソフトウェア開発, AWS
- 代表的な仕事:
  - IPSのシグネチャ設定を支援する機械学習の応用研究
  - 生成AIと機械学習を利用したVoC (Voice of Customers) 分析パイプライン処理の開発

github.com/HidetoshiKawaguchi | 𝕏 @Hidetoshi_RM | in linkedin.com/in/hidetoshi-kawaguchi-380a93281

本資料の注意事項

- 本資料は教育目的で公開しています
- 本資料はとある大学で学生向けに講義した内容を流用し作成されています
  - 大学からの希望があり、大学名は伏せています
  - 講義名は架空のものに置き換えています
- 資料内容はすべて川口個人の見解によるものであり、所属組織の公式見解と異なることがあることにご注意ください

前々回学んだこと

前々回は、データサイエンティストの成果である「ドキュメント」について、その背景と、それを書く上で基本的な考え方を学びました。

- ドキュメントコミュニケーションとは何か
- ビジネスで使われるドキュメント
- ドキュメント作成方法
- パラグラフ・ライティング

前回学んだこと

前回は、データサイエンティストの主要業務である、分析・実験・検証の方法論について学びました。

- 一般的なデータ分析プロセスのフレームワークである、CRISP-DMの紹介
- データサイエンティストの分析について解説
- 分析・実験・検証の管理方法の紹介

これまで学んだこと

これまでに学んだことは、データサイエンティストにとってメインとなる必須の方法論です。

- ドキュメント・コミュニケーション
- 分析・実験・検証の管理手法

適切に分析・実験・検証を行うことで知見を積み上げ、正しく伝わるドキュメントをまとめ社内外の他者に繋げる方法論を紹介した、ということになります。

これらを駆使することで、より実務で役立てるデータサイエンティストになることができます。

本日学ぶこと

更に一歩進んだデータサイエンティストとして活躍するための、ソフトウェアエンジニアリングから特に実践的かつ有効な方法論を学びます。

- Pythonコードの型・規約・自動整形
  読みやすいコードを書くために
- テストコード
  バグが少ないコードを書くために

なぜエンジニアリングについて学ぶのか

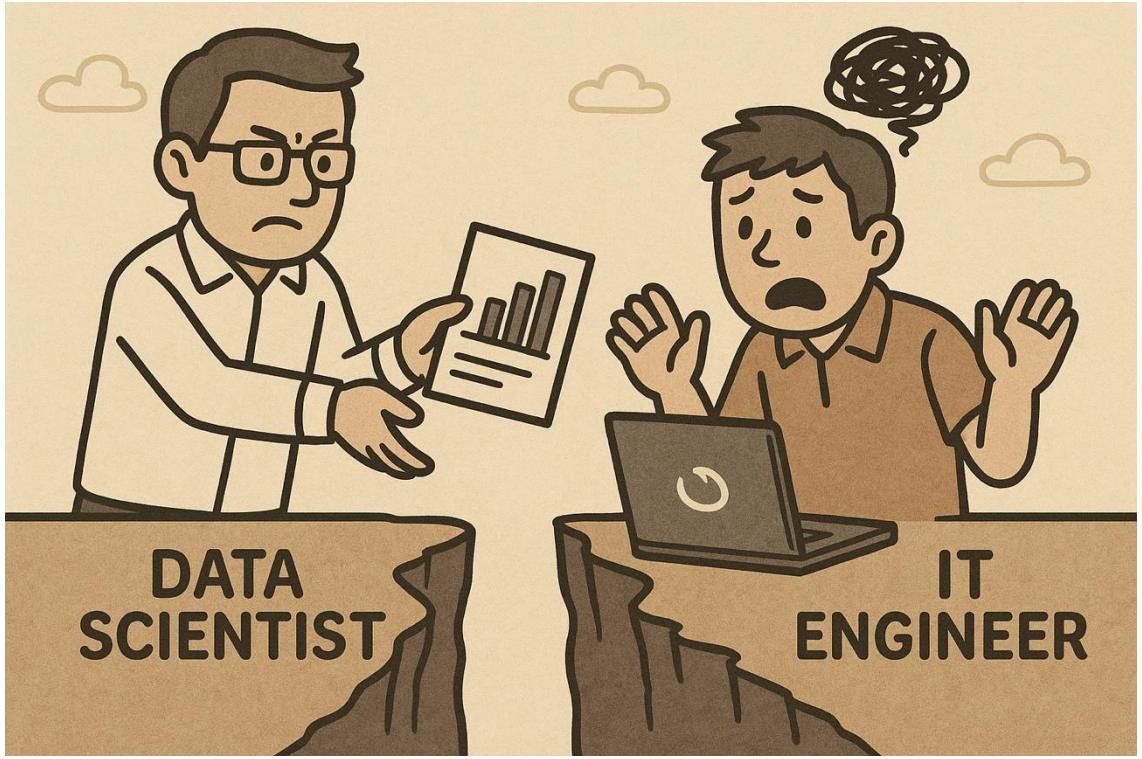
データサイエンティストの成果は最終的にはシステムやサービス、ソリューションといった何らかの形にする必要があるため、エンジニアリングのスキルもあると良いです。

- 知見があってもお客さんの元に届けるためのモノづくりが必要
- モノづくりを円滑に行うことで、最終的な目的を達成する

知見を形にする際の深い谷

データサイエンティストからITエンジニア(AI/機械学習エンジニア)に成果を渡して本格的な開発を始めるステップがあります。

異なる役割同士のコミュニケーションは大変です。



実際に起こった（起こり得る）問題

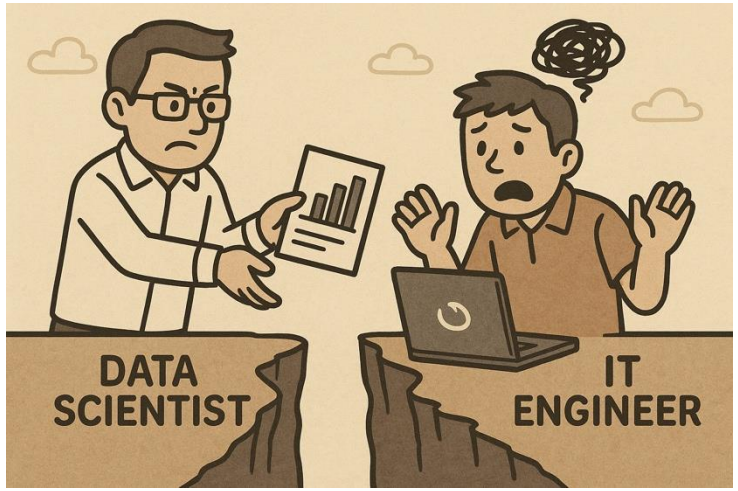
データサイエンティストのエンジニアリング不足によって、以下のような問題が起こります。

- **機械学習モデルのPoC (Proof of Concept)のコードが汚くてITエンジニアが解読困難**
- 開発スケジュールがタイトで、PoCのコードをほぼそのまま使う必要が出てくる
- データサイエンティストがシステムのコアな部分を担当しても、品質が低くバグが多発

エンジニアリングを学ぶことの必要性

このような問題を起こさないために有効な方法は、**データサイエンティストもエンジニアリングについて学んでおく、**ということです。

データサイエンティストがITエンジニアに丸投げして困らせないようにするためのスキルを学んでいきましょう。



こうならないように、データサイエンティストもエンジニアリングを学びましょう。

本日学ぶこと（再掲載）

更に一歩進んだデータサイエンティストとして活躍するための、ソフトウェアエンジニアリングから特に実践的かつ有効な方法論を学びます。

- **Pythonコードの型・規約・自動整形**
  読みやすいコードを書くために
- **テストコード**
  バグが少ないコードを書くために

Pythonの特徴

Pythonは動的型付けの言語であり、変数に型を明示的に宣言する必要がありません。

- メリット
  - 軽快にコーディングでき、以下の用途に向いている
    - プロトタイピング
    - データ解析のような探索的作業
- デメリット
  - 読みづらいコードになる
    - 変数の型がわからず、読み手が理解しづらい
    - IDEや静的解析ツールの補助が弱くなる
  - 実行時エラーになりやすい
    - 型の不一致がコンパイル時には検出されなく、実行して初めてエラーとして気づく

Pythonはすぐに動くが、長期的には読みづらく壊れやすい
→ 実は本格的な開発には工夫が必要です。

Pythonコードの型・規約・自動整形のポイント

- 型ヒント
  関数・メソッドの引数と返り値に型を書こう
- コーディング規約
  PEP8に代表されるコーディング規約に沿って書こう
- リンター
  解析ツールでコードの品質を確かめよう
- フォーマッター
  コードを綺麗にするツールは積極的に使おう

型ヒント

Pythonは変数の型を宣言する必要はありませんが、ヒントとして書くことはできます。

- 型ヒントがない場合

```
def add(a, b):
    return a + b
```

- 型ヒントがある場合

引数の型　　返り値の型

```
def add(a: int, b: int) -> int:
    return a + b
```

型ヒント導入のメリット

実際の開発で役立つたくさんのメリットがあります。**開発はもちろんのこと、分析・実験・検証用コードでも必須レベル**

- 読み手のコード理解を助ける
  - 自分も後で見返す時助かります
- コードエディタの補完に役立つ
- 静的解析ツールを使うことで、実行前にエラーに気づける

学生の皆さんは、研究の実験で使う場合、そのコードにも積極的に書いて慣れておくことをお勧めします。

色々な型ヒント (1)

基本的には全ての型に対応しています。

- strやfloat

```
def format_price(name: str, price: float, discount: float) -> str:
    return f"{name}: {price * (1 - discount)} 円"
```

- list (リスト)
  listの場合は、括弧 [] で要素の型を指定します。

```
def average_score(scores: list[float]) -> float:
    return sum(scores)/len(scores)
```

色々な型ヒント (2)

- dict (辞書型, 連想配列)
  括弧 [] でキーとバリューを指定します。

```
def get_score(student_scores: dict[str, float], name: str) -> float:
    if name in student_scores:
        return student_scores[name]
    else:
        return 0.0
```

- 複数(または)の型
  | (パイプ)で「または」の値、という表現ができます。
  以下の例は、返り値が float か str のいずれかです。

```
def get_score(student_scores: dict[str, float], name: str) -> float | str:
    return student_scores.get(name, "No Data")
```

色々な型ヒント (3)

- クラス・bool・None

```
# 独自のクラス定義
class Student:
    def __init__(self, name: str, score: float) -> None:
        self.name = name
        self.score = score

def is_passing(student: Student) -> bool:
    return student.score >= 60.0
```

- その他
  他にも色々な書き方があります。必要になった時都度ChatGPTに聞くかGoogleで調べましょう

型ヒントを使えるPythonのバージョン

型ヒント自体はPython 3.5から導入されましたが、新しいマイナーバージョン(最新だと3.13)が登場するたびに、新機能が追加されています。

前スライドのサンプルプログラムは、3.10以降対応です。

- 3.9以前は、list, dict, 「または」を使うために専用クラスをインポートする必要がありました。
- 3.11以降に導入された機能はややマニアック

コーディング規約

コーディング規約とは、**プログラムを書くときのルールや約束事をまとめたもの**です。

目的

- チーム全体で統一感のあるコードを書くため
- 可読性を高め、レビューや保守を楽にするため
- バグを減らし、品質を安定させるため

代表的な内容

- インデントの幅（Pythonはスペース4つが基本）
- 変数や関数の名前の付け方（スネークケース my_variable など）
- 行の長さ（PEP8では1行79文字が推奨）
- 空行やコメントの書き方
- インポート文の並び順

代表的なコーディング規約 PEP8

事実上のスタンダードであるコーディング規約に、**PEP8 (Python Enhancement Proposal 8)**というものがあります。

- Python公式が推奨するコーディング規約
- Pythonコードを「読みやすく」「統一的に」書くためのルールをまとめた文書
- Pythonを作った Guido van Rossum が執筆（2001年）し、その後もメンテナンスされてきた

PEP8の代表的なルール

PEP8のルールを以下に抜粋して紹介します。

- インデント: スペース4つ (タブは使わない)
- 行の長さ: 1行79文字
- スペース: =, +, - などの前後にスペースを入れる
- 空行：
  - 関数やクラス定義の前は2行空ける
  - 関数内では関連する処理を区切るために1行空ける
- 名前付け
  - 変数・関数名はスネークケース (例 user_name)
  - クラス名はキャメルケース (例 UserProfile)
  - 定数は 全て大文字 (例 MAX_RETRY)
- インポート
  - 標準ライブラリ → サードパーティ → 自作モジュールの順でまとめる
  - 1行に複数の import を書かない

他にもたくさんあります！

コーディング規約に関する提言

自分流の綺麗さより、チームのルールを優先すること

多くの企業ではPEP8をベースとしたルールを使っています。
（PEP8から一部のルールを無視したり追加して使っています）

学生のうちから、PEP8を意識したコードを書いておくと後で楽です。

ツールの勧め

- 型ヒント
- コーディング規約

これらの重要性は分かったけど、手動でのコードのチェックが大変すぎる・・・。

それらを支援するツールがあるので、それらでカバーできます。

コードの品質を高めるツール

コードの品質を高めるツールとして、以下の３つがあります。

- **静的型チェッカー**
  型ヒントに基づいてコードの「意味的な正しさ」を検証する
  - 代表例 mypy
- **リンター (Linter)**
  コードの規約違反や潜在的なバグを検出するツール
  - 代表例: flake8
- **フォーマッター (Formatter)**
  コードの見た目を自動整形するツール
  - 代表例: black, isort

mypyの紹介

mypy は、Python用の静的型チェッカーです。型ヒントに基づいて、実行前に型の不一致を検出できる。

- 動かしてみないとわからないエラーを事前に防げる
- チェックの厳しい strict モードもある

: my[py]

mypyの動作紹介

mypy_error_sample.py

```
1 def square(x: int) -> int:
2     return x * x
3 square("abc")
```

```
(class04-py3.11) % mypy mypy_error_sample.py
mypy_error_sample.py:3: error: Argument 1 to "square" has incompatible type "str"; expected "int"  [arg-type]
Found 1 error in 1 file (checked 1 source file)
```

実行しなくても、型が違うことを教えてくれる。

mypy --strictの動作紹介

--strict オプションを付与することでより厳格なチェックができる。

- 全ての引数・返り値の型ヒントを強制する
  → 型ヒントの付与忘れも検知できる
- オプションなしの場合、型ヒントはなくても良い。あるところだけチェックされる。

strict_mypy_error_sample.py

```
1 def square(x) -> int:
2     return x * x
3 square(3)
```

型ヒントがないことを指摘している

```
(class04-py3.11) % mypy --strict strict_mypy_error_sample.py
strict_mypy_error_sample.py:1: error: Function is missing a type annotation for one or more arguments  [no-untyped-def]
strict_mypy_error_sample.py:2: error: Returning Any from function declared to return "int"  [no-any-return]
Found 2 errors in 1 file (checked 1 source file)
```

flake8の紹介

flake8はリンターの一種で、主にコードがPEP8に準拠しているかをチェックしてくれます（それだけではない）

flake8の動作紹介

flake8_sample.py

```python
import math, sys
import os

def add(a,b):
  return a+b


def long_function_name(var_one, var_two, var_three, var_four, var_five,
                       var_six, var_seven, var_eight, var_nine, var_ten):
    print(var_one)
```

```
(class04-py3.11) % flake8 flake8_sample.py
flake8_sample.py:1:1: F401 'math' imported but unused
flake8_sample.py:1:1: F401 'sys' imported but unused
flake8_sample.py:1:12: E401 multiple imports on one line
flake8_sample.py:2:1: F401 'os' imported but unused
flake8_sample.py:4:1: E302 expected 2 blank lines, found 1
flake8_sample.py:4:10: E231 missing whitespace after ','
flake8_sample.py:5:3: E111 indentation is not a multiple of 4
flake8_sample.py:9:80: E501 line too long (100 > 79 characters)
```

Blackの紹介

Black は、Python用の代表的なフォーマッターです。

- コードスタイルを自動で統一する
- 基本的にはPEP8に準拠
  （１行当たりの文字数やインポートの順番は変えない）
- 設定が少ない
  -> 「自分流」ではなく「チーム全員が同じスタイル」になる

Blackの動作紹介

black_before.py

```
def process_data(data:list[int],limit:int=10,verbose:bool=False)->dict[str,int]:
 if verbose: print("processing...");   result={"count":0,"sum":0}
 for i in data: result["count"]+=1;result["sum"]+=i
 if result["count"]>limit: print("too many items")
 return result
```

```
(class04-py3.11) % black  black_before.py
reformatted black_sample.py

All done! ✨ 🍰 ✨
1 file reformatted.
```

black_after.py

```
def process_data(
    data: list[int], limit: int = 10, verbose: bool = False
) -> dict[str, int]:
    if verbose:
        print("processing...")
        result = {"count": 0, "sum": 0}
    for i in data:
        result["count"] += 1
        result["sum"] += i
    if result["count"] > limit:
        print("too many items")
    return result
```

Black の注意

ほぼPEP8には準拠しているが、これだけで準拠しきれません。

- 1行あたりのデフォルトの文字数
  - PEP8は79文字
  - Blackは88文字（設定で変更できる）
- パッケージのインポート順序を変更しない

実践的な工夫は後で紹介します。

isortの紹介

import 文を自動でソート・整形してくれるフォーマッター

- PEP8準拠
- アルファベット順に並び替え
- import の種類ごとにグループ分け
  - 標準ライブラリ
  - サードパーティライブラリ（pip でインストールしたもの）
  - 自作モジュール
- 余分なスペースや順序の乱れも修正

isort の動作紹介

isort_before.py

```
1 import pandas as pd
2 import os, sys
3 from hoge import fuga
4 import numpy as np
```

```
(class04-py3.11) % isort  isort_before.py
Fixing /Users/kawaguchi/repositories/kanagawa-uc/class04/  isort_before.py
```

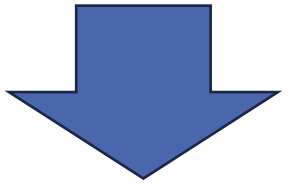
isort_after.py

```
1 import os
2 import sys
3
4 import numpy as np
5 import pandas as pd
6
7 from hoge import fuga
```

実践的なツールの使い方

コードを一通り書いた後に、以下の流れを実行するのが鉄板

- Black と isort を適用して整える
- flake8 と mypyでチェックし、指摘があれば修正する



これでかなり他の人にとっても読みやすいコードになる。

Blackとisortの設定

BlackとisortでPEP8に準拠するようにフォーマットするには設定が必要です。

以下の設定を pyproject.tomlに追記し、isort -> Blackの順で適用
（使用している仮想環境管理ツールにより記法が異なります）

```
[tool.black]
line-length = 79  # PEP8準拠

[tool.isort]
profile = "black" # Blackと衝突しないように
line_length = 79  # PEP8準拠
```

一歩進んだ設定

フォーマッターとリンターを、自動実行するように設定することもできますが、詳細は割愛します。興味がある人は、ツールの使い方も含めて調べるかChatGPTに聞いてみましょう。

- Gitの pre-commitフック, pre-pushフックでローカルで実行
- Github の Github Actionsでサーバで実行

ツールを使うことの注意点

フォーマッターとリンターの利用は便利で即効性がありますが、**そもそもコードを綺麗に構造化しておくことが何より大事**

- オブジェクト指向プログラミング
- 関数型プログラミング

これらの考え方を身につけておき、整理されたコードを書けるように学生のうちから練習しておきましょう。

Pythonコードの型・規約・自動整形のポイント（再掲載）

- 型ヒント
  関数・メソッドの引数と返り値に型を書こう
- コーディング規約
  PEP8に代表されるコーディング規約に沿って書こう
- リンター
  解析ツールでコードの品質を確かめよう
- フォーマッター
  コードを綺麗にするツールは積極的に使おう

テストコードとは

テストコードとは、プログラムが正しく動いているかを確認するためのコード

- プログラム本体には組み込まれない
- 目的はあくまで「動作確認」
- 人間が手作業で試す代わりに、自動でチェックしてくれる

テストコードの例

例えば、以下のようなコードでは、足し算を行う add 関数の動作をテストしている。

```
1 def add(a: int, b: int) -> int:
2     return a + b
3
4 # テストコード
5 def test_add():
6     assert add(2, 3) == 5
```

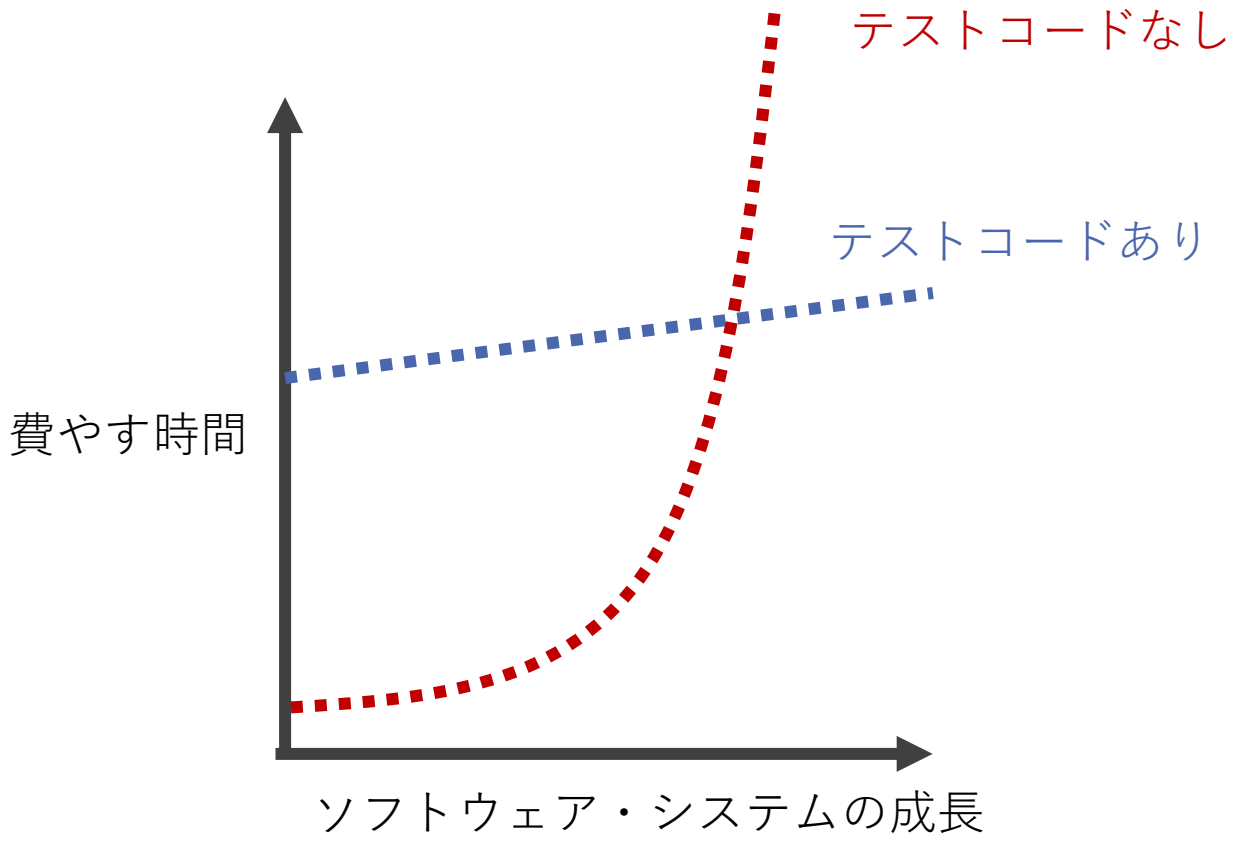
テストコードの重要性

分析・研究をメインに活動していると実感しづらいですが、
本格的なソフトウェア・システム開発において、
テストコードは品質を示す上で極めて重要であり書くことは必須です。

- コード変更で「知らないうちに壊した」を防ぐ
- 使用どおりに動くことを証明する
- 安心してリファクタリングできる
- コードの使い方（例）を示すドキュメントにもなる

テストコードを書く場合と書かない場合

テストコードを書かない場合、ソフトウェア・システムの成長に伴い、開発に費やす時間が指数関数的に増えていきます。



テストコードを書く労力

極めて重要でありながら、テストコードを書くことはすごく大変です。
データサイエンティストは、**ITエンジニアがテストコードを書く負担を軽視しないようにしましょう**。

テストコードを書く負担は、テスト対象の本体コードを書くことの1~10倍大変と言われている。（参考図書: 単体テストの考え方/使い方）

つまり例えば、あなたが1日かけて作ったコードをテストするためのコードは、プラスで1日～10日かかる

※ 10倍は流石に珍しいですが、本当に大変なことがあります・・・。

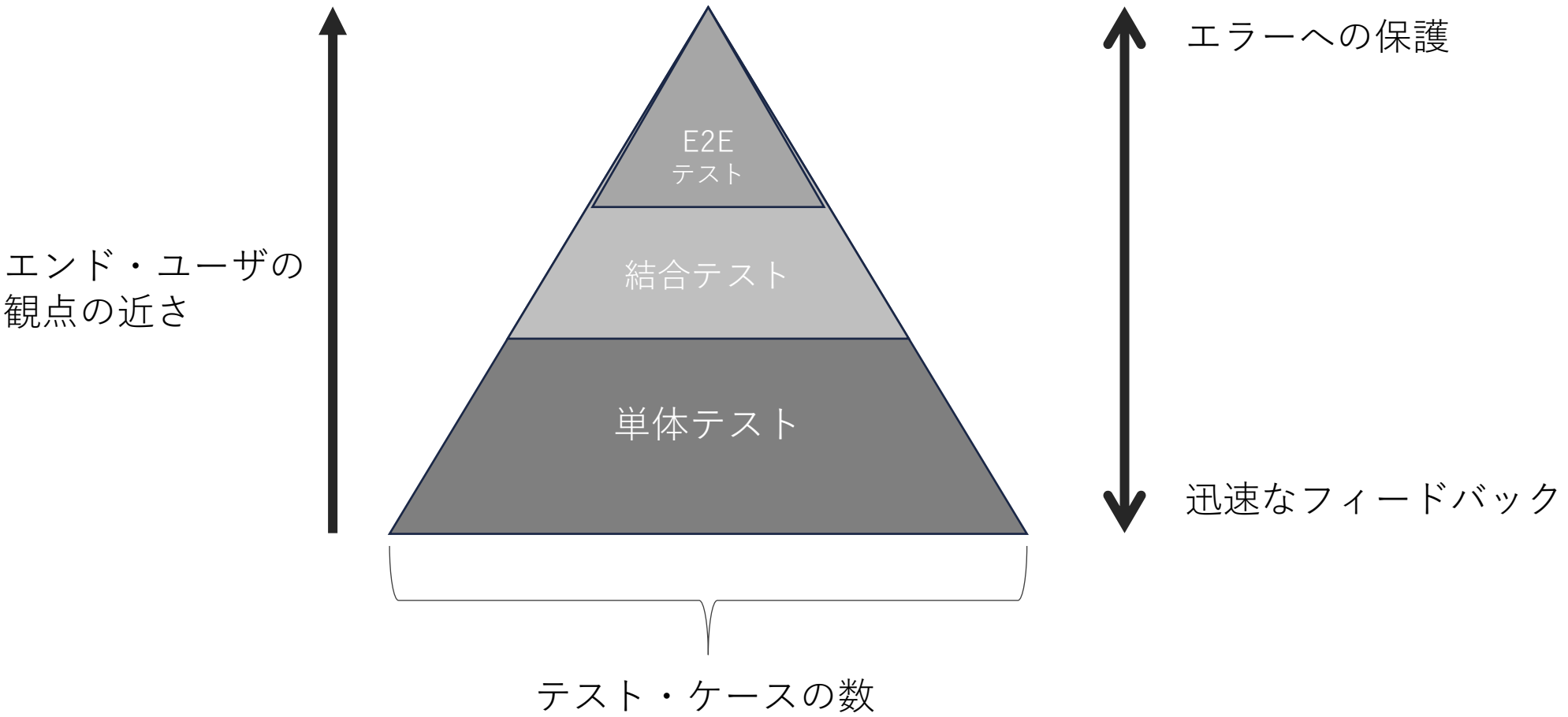
テストコードの種類

大きく分けて以下の３種類のテストがあります

- **単体テスト（Unit Test）**
  - 最小の部品（関数・クラス）を対象
  - 狙い: 小さなバグを早期に発見
- **結合テスト（Integration Test）**
  - 複数の部品を組み合わせた処理を対象
  - 狙い: モジュール間のやりとりが正しいか確認
- **E2Eテスト（End-toEnd Test）**
  - ソフトウェア・システム全体を通して動かすテスト
  - ユーザ視点で全体が期待通りか確認

テスト・ピラミッド

単体テスト〜E2Eテストの関係を示すと、単体テストほどテスト・ケースの数が多くフィードバックが迅速、E2Eテストはエンド・ユーザに近く、エラーへの保護能力が高い



（参考）日本のSIer文化におけるテスト

日本のSIer では、結合テストとE2Eテストの間に、システムテスト、というものがあります。

全体を確認する結合テスト、くらいの意味です。

この講義の題材

この講義では、以下の理由により**単体テストを題材**とします。

- 最も基礎的なテスト
  - 単体テストを理解できれば、結合テストやE2Eテストにも応用できる
- 範囲が小さく理解しやすい
- すぐ効果を実感できる
- 習慣化しやすい
- 設計の甘さに気づける
- 現場でも重視される

使うテストフレームワーク: pytest

pytestとは 最も普及しているPythonのテストフレームワークです。

- シンプル: ただ関数を書いて assert するだけ
- 自動検出: test_ で始まる関数やファイルを自動で見つける
- 豊富な機能
  - 複数ケースを一気にテスト
  - 前処理・後処理を簡単に共通化
  - 一時フォルダの利用も簡単
- エラーメッセージがわかりやすい

pytest

pytestのルール: フォルダ構造

pytestでは、以下のフォルダ構造を基本とします。ここでは、mypackageというモジュール名で、その中に、mathutils.pyという本体コードがある前提とします

- mypackage/
  - mathutils.py
- tests/
  - test_mathutils.py
  - __init__.py

本体コード

テストコード
- ファイル名や関数名の規則に従ってpytestがテストであることを自動検出する
- __init__.py はおまじない

フォルダのルートにtestsディレクトリを置き、本体とは完全に独立させることがポイント

pytestの実行例 –前提-

以下のフォルダ構成でルートディレクトリで実行するとします。

```
(mypackage-py3.11) % tree
.
├── mypackage
│   └── mathutils.py
├── poetry.lock
├── pyproject.toml
└── tests
    ├── __init__.py
    └── test_mathutils.py

3 directories, 5 files
```

mathutils.py

```python
def add(a: int, b: int) -> int:
    # 足し算（下のコードは正しい）
    return a + b

def sub(a: int, b: int) -> int:
    # 引き算（下のコードは間違い）
    return a * b
```

test_mathutils.py

```python
from mypackage.mathutils import add, sub

def test_add() -> None:
    assert add(3, 2) == 5

def test_sub() -> None:
    assert sub(3, 2) == 1
```

成功するテストケース

失敗するテストケース

pytestの実行例 –実行-

以下の動作実行例では、

- add関数の動作はOK
- sub関数の動作はNG
  - 3-2=1の値となることをテストケースでは期待している
  - しかし、sub関数の中身にバグがあり、6が返ってきている

```
(mypackage-py3.11)  % pytest
============================================================================ test session starts ============================================================================
platform darwin -- Python 3.11.3, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/kawaguchi/repositories/kanagawa-uc/mypackage
configfile: pyproject.toml
collected 2 items

tests/test_mathutils.py .F                                                                                                                                                [100%]

================================================================================= FAILURES =================================================================================
_____________________________________________________________________________ test_sub _____________________________________________________________________________

    def test_sub() -> None:
>       assert sub(3, 2) == 1
E       assert 6 == 1
E        +  where 6 = sub(3, 2)

tests/test_mathutils.py:9: AssertionError
========================================================================== short test summary info ==========================================================================
FAILED tests/test_mathutils.py::test_sub - assert 6 == 1
============================================================================= 1 failed, 1 passed in 0.01s ==============================================================================
```

コードのバグをテスト失敗として検出

pytestの基本ルール

pytestでは、実行時に以下のルールで全てのコードからテストを自動で検出し、assert で判定を行う。

テストコードの検出

- test_*.pyまたは *_test.py というファイル名
- これらの中の test_ で始まる関数を自動実行
- クラスの場合は Test で始まるクラスの中の test_* メソッドを実行

テストの判定

- assert で真偽値判定 (Trueなら成功, Falseなら 失敗)
  - assert 自体は pythonの標準機能（デバッグ・テスト用）
  - pytest以外で assert が Falseの時は AssertionError という例外をあげる

単体テストの基本的な考え方

前提となるインプット（引数）と期待するアウトプット（返り値）のペアを用意しておき、実装した関数の結果と比較して正しいかを確認する。

先の例だと、以下の考えをコードとして書いている。

- 「add 関数の場合は、3 と 2を引数とすると 5が返るはず」
- 「sub 関数の場合は、3 と 2を引数とすると 1が返るはず」

テストコードを書かないとき、皆さんも関数の動作確認をするときに同様に考えているはずです。それをそのままコードで実行できるようにします。

pytestの機能を一部紹介

- 同じテストで複数のパラメータを実行する parametrize
- テスト実行前に必要な準備を自動で行ってくれる仕組み fixture

同じテストで複数のパラメータを実行する parametrize

parametrize という機能を使えば、複数のパラメータでのテストを簡単に実行できる。

以下のコードでは、変数a, b, expected という3つの変数（引数）に、３パターンの値の組み合わせを試している。

```python
import pytest
from mypackage.mathutils import add

@pytest.mark.parametrize(
    "a, b, expected",
    [
        (3, 2, 5),
        (0, 0, 0),
        (-1, 1, 0),
    ],
)
def test_add(a: int, b: int, expected: int) -> None:
    assert add(a, b) == expected
```

テスト実行前に必要な準備を自動で行ってくれる仕組み fixture

fixtureで複数のテストで使う処理簡単に記述できます。

以下の例では、numbersをfixtureとして定義し、各々のテスト関数で実行前に関数として実行され、変数に格納されます。

test_fixture.py

```python
import pytest

@pytest.fixture
def numbers() -> list[int]:
    return [1, 2, 3, 4]

def test_sum(numbers: list[int]):
    assert sum(numbers) == 10

def test_max(numbers: list[int]):
    assert max(numbers) == 4
```

※ ここでは動作紹介のために、標準関数の sum, maxを使っています。

そのほかのpytestの便利な機能

- conftest.py
  - プロジェクト全体で共有するfixtureや設定を書く特別ファイル
  - import なしで自動的に認識される
- 組み込みfixture (デフォルトで備わっているfixture)
  - tmp_path/tmpdir: 毎回クリーンな一時ディレクトリを提供
  - capsys/cafd: stdout/stderrの出力をキャプチャ
  - monkeypatch: 環境変数や属性を一時的に変更
- マーカー機能: タグ付けのような機能
  - 実行時に特定のマーカーだけついたテストだけ実行のような使い方
- 例外のテスト
- プラグイン機能

調べると大抵の機能はあります。

奥深いテストの世界

テストを極めるだけで食いっぱぐれないです

それくらい奥深い世界なので、興味がある人は参考書籍で学んでみましょう。

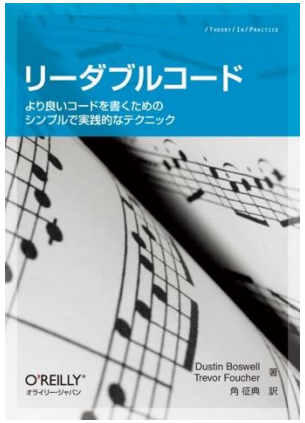
本日学んだこと

更に一歩進んだデータサイエンティストとして活躍するための、ソフトウェアエンジニアリングから特に実践的かつ有効な方法論を学びます。

- **Pythonコードの型・規約・自動整形**
  読みやすいコードを書くために
- **テストコード**
  バグが少ないコードを書くために

おすすめ参考書籍

- **リーダブルコード**—より良いコードを書くためのシンプルで実践的なテクニック
  - Dustin Boswell、Trevor Foucher 著、角 征典 訳
  - オライリー
  - ロングセラー。薄くて読みやすい。１回は目を通すこと



- **単体テストの考え方/使い方** プロジェクトの持続可能な成長を実現するための戦略
  - Vladimir Khorikov 著、須田 智之 訳
  - マイナビブックス
  - 単体テストだけじゃなくて、テストの考え方がよくわかる

演習・課題

コードの修正とテストコードの作成にチャレンジしてもらいます。

- 提出方法: 学内システムのレポート機能
- 締め切り: XXX
- 課題の内容

学内システムからZipファイル (class09_exercise.zip)をダウンロードし、その中にあるコードを、以降のスライドの演習内容に応じて編集して保存し、提出すること。

- 備考
  - 作業はZipファイルを展開してできたフォルダ内で行う
  - 最終的にはできたコードは、データやログとまとめてzipに圧縮して保存・提出すること

演習1: Pythonコードの型・規約・自動整形

コードを編集して、以下の2つのコマンドをパスするようにすること

- `flake8 <ファイル名>`
- `mypy --strict <ファイル名>`

補足

- isort, や blackは使っても使わなくても良い
- プログラムの仕様を修正することは禁止

演習1-1

exercise_1_1.py

```
def add(a, b):
  return a+b
```

演習1-2

exercise_1_2.py

```
import math, sys

def discounted(price, rate = 0.1):
    if rate<0 or rate>1:print("invalid")
    return price*(1-rate)
```

演習1-3

exercise_1_3.py

```
import math, sys

def parse_prices(prices):
    result = {}
    for k,v in prices.items():
        try:
            if isinstance(v,str): x=float(v)
            else: x=float(v)
        except Exception as e:
            x=0.0
        result[k]=x
    return result

def stats(nums):
    n=len(nums);
    if n==0: return {"count":0,"mean":0.0,"stdev":0.0}
    mean=sum(nums)/n
    s=0.0
    for x in nums: s+= (x-mean)**2
    stdev= math.sqrt(s/n) if n>1 else 0.0
    return {"count":n,"mean":mean,"stdev":stdev}

class User:
    def __init__(self,name,scores:list[float]):
        self.name=name; self.scores=scores
    def avg(self):
        if not self.scores: return 0.0
        return sum(self.scores)/len(self.scores)
```

演習2

関数の下にテストを追記して、pytest で実行しパスすることを確かめましょう。

演習2-1: 偶奇判定関数

関数 is_evalをテストするコード、test_is_evenを以下の条件で完成させよ。

- is_even関数は、与えられた整数値が偶数ならTrueを、奇数ならFalseを返す関数
  - 例えば 2なら True, 1ならFalseを返す
- parametrizeを使って複数の整数入力に対して正しい結果になるか確認すること
- 正の数・負の数・0なども含めること

確認用コマンド
pytest exercise_2_1_test.py

exercise_2_1_test.py

```
def is_even(n: int) -> bool:
    return n % 2 == 0

def test_is_even(...) -> None:
    ...
```

演習2-2: 文字列の反転

関数 reverse_stringをテストするコード、test_reverse_stringを以下の条件で完成させよ。

- reverse_stringは、与えられた文字列(str)を反転して返す関数
  - 例えば “ABCD”を与えられたなら、”DCBA”を返す
- parametrize を使い、空文字・1文字・複数文字を含む複数のケースを網羅すること。

確認用コマンド
pytest exercise_2_1_test.py

exercise_2_2_test.py

```
def reverse_string(s: str) -> str:
    return s[::-1]

def test_reverse_string(...) -> None:
    ...
```

演習2-3: テキストファイルの行数カウント

関数 count_linesをテストするコード、test_count_linesを以下の条件で完成させよ。

- テキストファイルはテストコードの中で作成・保存し、テスト完了後に削除すること
  - わかる人は組み込みfixtureのtmp_pathかtmpdir を使うと簡単
- 最低限、以下の 3 種類のテキストファイルをテストすること
  - 空のテキストファイル
  - 1行だけのテキストファイル
  - 2行以上のテキストファイル

確認用コマンド
pytest exercise_2_1_test.py

exercise_2_3_test.py

```
def count_lines(path: str) -> int:
    with open(path, "r", encoding="utf-8") as f:
        return sum(1 for _ in f)

def test_count_lines(...) -> None:
    ...
```

まとめ

これまでに学んだは、データサイエンティストの実践的なノウハウです。

- ドキュメント・コミュニケーション
- 分析・実験・検証の管理手法
- コーディング規約・テストコード

適切に分析・実験・検証を行うことで知見を積み上げ、正しく伝わるドキュメントをまとめ社内外の他者に繋げる、信頼できるソフトウェア・システムを作るための必須ノウハウです。

データサイエンティストになる人にとって、就職後にこれまで学んだことが少しでも活きると幸いです

Copyright © 2025 Hidetoshi Kawaguchi